In [ ]:
```
!jupyter nbconvert --to html d6_More_Functional_API.ipynb
```

In [1]:
```
import pandas as pd # Use pandas to load dataset from csv file
import os
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers
from tensorflow.keras.datasets import mnist
```

In [2]:
```
BATCH_SIZE = 64
WEIGHT_DECAY = 0.001
LEARNING_RATE = 0.001

train_df = pd.read_csv("train.csv")
test_df = pd.read_csv("test.csv")
train_images = os.getcwd() + "/train_images/" + train_df.iloc[:,0].values
test_images = os.getcwd() + "/test_images/" + test_df.iloc[:,0].values

train_labels = train_df.iloc[:,1:].values
test_labels = test_df.iloc[:,1:].values
```

In [3]:
```
def read_images(image_path, label):
    image = tf.io.read_file(image_path)
    image = tf.image.decode_image(image, channels=1, dtype=tf.float32)

    image.set_shape((64,64,1))
    label[0].set_shape([])
    label[1].set_shape([])

    labels = {"first_num": label[0], "second_num": label[1]}
    return image, labels

AUTOTUNE = tf.data.experimental.AUTOTUNE
train_dataset = tf.data.Dataset.from_tensor_slices(
    (train_images, train_labels)
)
train_dataset = (
     train_dataset.shuffle(buffer_size=len(train_labels))
    .map(read_images)
    .batch(batch_size=BATCH_SIZE)
    .prefetch(buffer_size=AUTOTUNE)
)

test_dataset = tf.data.Dataset.from_tensor_slices((test_images, test_labels))
test_dataset = (
    test_dataset.map(read_images)
    .batch(batch_size=BATCH_SIZE)
    .prefetch(buffer_size=AUTOTUNE)
)
```

In [4]:
```
inputs = keras.Input(shape=(64,64,1))
x = layers.Conv2D(
    filters=32,
```

```
    kernel_size=3,
    padding="same",
    kernel_regularizer=regularizers.l2(WEIGHT_DECAY),
)(inputs)

x = layers.BatchNormalization()(x)
x = keras.activations.relu(x)

#1. Conv2D
x = layers.Conv2D(64,3,kernel_regularizer=regularizers.l2(WEIGHT_DECAY))(x)
x = layers.BatchNormalization()(x)
x = keras.activations.relu(x)
x = layers.MaxPooling2D()(x)

#2. Conv2D
x = layers.Conv2D(128, 3, activation="relu")(x)
x = layers.MaxPooling2D()(x)
x = layers.Flatten()(x)
x = layers.Dense(128, activation="relu")(x)
x = layers.Dropout(0.5)(x)
x = layers.Dense(64, activation="relu")(x)
output1 = layers.Dense(10, activation="softmax", name="first_num")(x)
output2 = layers.Dense(10,activation="softmax", name="second_num")(x)
model = keras.Model(inputs=inputs, outputs = [output1, output2])

model.compile(
    optimizer=keras.optimizers.Adam(LEARNING_RATE),
    loss=keras.losses.SparseCategoricalCrossentropy(),
    metrics=["accuracy"],
)

model.fit(train_dataset, epochs=5, verbose=2)
model.evaluate(test_dataset, verbose=2)
```

```
Epoch 1/5
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
Cell In[4], line 35
     27 model = keras.Model(inputs=inputs, outputs = [output1, output2])
     29 model.compile(
     30     optimizer=keras.optimizers.Adam(LEARNING_RATE),
     31     loss=keras.losses.SparseCategoricalCrossentropy(),
     32     metrics=["accuracy"],
     33 )
---> 35 model.fit(train_dataset, epochs=5, verbose=2)
     36 model.evaluate(test_dataset, verbose=2)

File ~\miniconda3\envs\myenv\lib\site-packages\keras\src\utils\traceback_utils.py:65, in filter_traceback.<locals>.error_handler(*args, **kwargs)
     63 filtered_tb = None
     64 try:
---> 65     return fn(*args, **kwargs)
     66 except Exception as e:
     67     filtered_tb = _process_traceback_frames(e.__traceback__)

File ~\miniconda3\envs\myenv\lib\site-packages\keras\src\engine\training.py:1742, in Model.fit(self, x, y, batch_size, epochs, verbose, callbacks, validation_split, validation_data, shuffle, class_weight, sample_weight, initial_epoch, steps_per_epoch, validation_steps, validation_batch_size, validation_freq, max_queue_size, workers, use_multiprocessing)
   1734 with tf.profiler.experimental.Trace(
   1735     "train",
   1736     epoch_num=epoch,
   (...)
   1739     _r=1,
   1740 ):
   1741     callbacks.on_train_batch_begin(step)
-> 1742     tmp_logs = self.train_function(iterator)
   1743     if data_handler.should_sync:
   1744         context.async_wait()

File ~\miniconda3\envs\myenv\lib\site-packages\tensorflow\python\util\traceback_utils.py:150, in filter_traceback.<locals>.error_handler(*args, **kwargs)
    148 filtered_tb = None
    149 try:
--> 150   return fn(*args, **kwargs)
    151 except Exception as e:
    152   filtered_tb = _process_traceback_frames(e.__traceback__)

File ~\miniconda3\envs\myenv\lib\site-packages\tensorflow\python\eager\polymorphic_function\polymorphic_function.py:825, in Function.__call__(self, *args, **kwds)
    822 compiler = "xla" if self._jit_compile else "nonXla"
    824 with OptionalXlaContext(self._jit_compile):
--> 825   result = self._call(*args, **kwds)
    827 new_tracing_count = self.experimental_get_tracing_count()
    828 without_tracing = (tracing_count == new_tracing_count)

File ~\miniconda3\envs\myenv\lib\site-packages\tensorflow\python\eager\polymorphic_function\polymorphic_function.py:890, in Function._call(self, *args, **kwds)
    886     pass  # Fall through to cond-based initialization.
    887   else:
```

```
    888     # Lifting succeeded, so variables are initialized and we can run the
    889     # no_variable_creation function.
--> 890     return self._no_variable_creation_fn(*args, **kwds)
    891 else:
    892   _, _, filtered_flat_args = (
    893       self._variable_creation_fn._function_spec  # pylint: disable=protected
-access
    894       .canonicalize_function_inputs(
    895           args, kwds))

File ~\miniconda3\envs\myenv\lib\site-packages\tensorflow\python\eager\polymorphic_f
unction\tracing_compiler.py:148, in TracingCompiler.__call__(self, *args, **kwargs)
    145 with self._lock:
    146   (concrete_function,
    147    filtered_flat_args) = self._maybe_define_function(args, kwargs)
--> 148 return concrete_function._call_flat(
    149     filtered_flat_args, captured_inputs=concrete_function.captured_inputs)

File ~\miniconda3\envs\myenv\lib\site-packages\tensorflow\python\eager\polymorphic_f
unction\monomorphic_function.py:1349, in ConcreteFunction._call_flat(self, args, cap
tured_inputs)
   1345 possible_gradient_type = gradients_util.PossibleTapeGradientTypes(args)
   1346 if (possible_gradient_type == gradients_util.POSSIBLE_GRADIENT_TYPES_NONE
   1347     and executing_eagerly):
   1348   # No tape is watching; skip to running the function.
-> 1349   return self._build_call_outputs(self._inference_function(*args))
   1350 forward_backward = self._select_forward_and_backward_functions(
   1351     args,
   1352     possible_gradient_type,
   1353     executing_eagerly)
   1354 forward_function, args_with_tangents = forward_backward.forward()

File ~\miniconda3\envs\myenv\lib\site-packages\tensorflow\python\eager\polymorphic_f
unction\atomic_function.py:196, in AtomicFunction.__call__(self, *args)
    194 with record.stop_recording():
    195   if self._bound_context.executing_eagerly():
--> 196     outputs = self._bound_context.call_function(
    197         self.name,
    198         list(args),
    199         len(self.function_type.flat_outputs),
    200     )
    201   else:
    202     outputs = make_call_op_in_graph(self, list(args))

File ~\miniconda3\envs\myenv\lib\site-packages\tensorflow\python\eager\context.py:14
57, in Context.call_function(self, name, tensor_inputs, num_outputs)
   1455 cancellation_context = cancellation.context()
   1456 if cancellation_context is None:
-> 1457   outputs = execute.execute(
   1458       name.decode("utf-8"),
   1459       num_outputs=num_outputs,
   1460       inputs=tensor_inputs,
   1461       attrs=attrs,
   1462       ctx=self,
   1463   )
   1464 else:
```

```
   1465   outputs = execute.execute_with_cancellation(
   1466       name.decode("utf-8"),
   1467       num_outputs=num_outputs,
   (...)
   1471       cancellation_manager=cancellation_context,
   1472   )

File ~\miniconda3\envs\myenv\lib\site-packages\tensorflow\python\eager\execute.py:5
3, in quick_execute(op_name, num_outputs, inputs, attrs, ctx, name)
     51 try:
     52   ctx.ensure_initialized()
---> 53   tensors = pywrap_tfe.TFE_Py_Execute(ctx._handle, device_name, op_name,
     54                                       inputs, attrs, num_outputs)
     55 except core._NotOkStatusException as e:
     56   if name is not None:

KeyboardInterrupt:
```

In [ ]: